

Management Review Agent

Day 1 - 本地模型、Docker 與需求規格教學手冊

適用環境：Windows + Cursor Bash + Docker Compose + Ollama

Day 1 核心成果：你已完成地端模型推論、GPU 驗證、Docker volume 與 localhost 限制，並建立需求、KPI 與 AI Skill 的規格文件。這份手冊用來複習『為什麼這樣做』與『遇到錯誤如何判斷』。

目前練習環境紀錄

項目	已驗證內容
模型	qwen2.5:7b · 約 4.7 GB
模型 digest	845dbda0ea48ed749caafd9e6037047aa19acfcfd82e704d7ca97d631a0b697e
推論 endpoint	http://127.0.0.1:11434
GPU	NVIDIA GeForce RTX 3060 Ti · 8 GB VRAM
容器	local-ollama · port 僅綁定 localhost
持久化	ollama_data volume 對應 /root/.ollama

版本：Day 1 / 2026-09-07

1. Day 1 的全貌

Day 1 的目的不是『把 AI 聊天模型跑起來』而已，而是證明模型、服務、容器、GPU、資料位置與網路邊界都可被追查。企業環境最怕的是看似能回答問題，卻不知道它用到哪個模型、資料是否外傳、重啟後是否消失。

產物	它證明什麼
local_model_checklist.md	模型、endpoint、GPU、volume、網路設定與驗收結果可追溯。
problem_statement.md	系統服務誰、協助什麼決策、哪些事情不做。
knowledge/kpi_dictionary.md	每個 KPI 的公式、單位、資料 owner 與例外口徑一致。
management_policy.md	異常門檻、資料例外、權限與安全規則可被實作。
skills/management_review/SKILL.md	AI 能力的輸入、輸出、限制、流程與驗收合約。
acceptance_cases.md	正常與失敗情境都有可重跑的預期行為。

最重要的設計原則

數字交給 Python 或 SQL 計算；LLM 負責理解問題、選擇受限 Tool、組織文字。 LLM 不能自由心算營收、成本率或離職率，也不能把『同期間有事件』說成『事件已證明是根因』。

2. 四個容易混淆的模型階段

階段	意思	本週對應
下載模型	把既有模型權重放到本機或容器的模型儲存區。	docker compose exec ollama ollama pull qwen2.5:7b
推論	向已運行的模型服務送 prompt，取得文字或結構化輸出。	Python POST /api/generate
微調	用訓練資料修改模型行為或權重。	Day 6：只做流程與評估設計
匯入與部署	將相容模型或 adapter 放進可服務的環境。	確認基底模型、架構與格式相容
常見誤解：下載 qwen2.5:7b 不等於訓練過它；Docker 裡有模型也不代表 Python 一定連到它；Modelfile 的 SYSTEM 提示詞是執行期設定，不等於改變模型權重。		

3. Docker、Ollama 與模型資料位置

Docker container 是執行中的隔離環境。container 被刪除時，container 內未持久化的檔案會消失；Docker volume 則是獨立保存的資料區。你的 compose 設定將 ollama_data 掛載到 /root/.ollama，因此模型下載後應保留在 volume 中。

```
services:
  ollama:
    ports:
      - "127.0.0.1:11434:11434"
    volumes:
      - ollama_data:/root/.ollama
    environment:
      - OLLAMA_NO_CLOUD=1
    restart: unless-stopped
```

設定	用途	不能單獨證明的事
127.0.0.1:11434	只允許本機連到 API，避免無限制暴露。	不代表整台電腦完全沒有外網連線。
ollama_data volume	容器重啟或重建後仍保留模型資料。	不代表所有應用資料都已持久化。
OLLAMA_NO_CLOUD=1	關閉 Ollama cloud 功能。	不代表其他套件、Embedding 或追蹤服務不會連外。
restart: unless-stopped	Docker 重啟後自動重啟服務。	不代表應用層健康檢查一定通過。

4. Cursor Bash 指令速查

若 Cursor 的終端顯示 MINGW64，通常是 Git Bash，Windows C 槽會對應到 /c。若使用 WSL，路徑通常是 /mnt/c。以下以 Git Bash 為例。

切換專案與檢查服務

```
cd "/c/Users/huang/Desktop/健鼎面試/management-review-agent"
docker compose ps
docker compose exec ollama ollama list
docker compose exec ollama ollama ps
```

執行 Python 推論驗證

```
# 不需 activate，也可直接使用專案虛擬環境的 Python
./venv/Scripts/python.exe verify_local.py
```

檢查 GPU

```
nvidia-smi
nvidia-smi --query-gpu=name,memory.total,driver_version --format=csv,noheader
```

查看 API 裡的模型與完整 digest

```
curl -s http://127.0.0.1:11434/api/tags
```

驗證重啟後模型仍可用

```
docker compose restart ollama
docker compose exec ollama ollama list
./venv/Scripts/python.exe verify_local.py
```

判讀 ollama ps：看到 qwen2.5:7b 與 PROCESSOR = 100% GPU，才可證明推論實際使用 GPU。只有 nvidia-smi 顯示 GPU 有記憶體使用量，不能證明該記憶體一定是 Ollama。

5. 這次實際遇到的 endpoint 問題

你曾遇到 Python 顯示『成功呼叫』，但之後因 result['response'] 發生 KeyError。問題不在 Python 讀取 JSON 的語法本身，而在於程式把『收到 HTTP 回覆』誤判成『推論成功』。

觀察到的現象	真正含義
容器 local-ollama 有 qwen2.5:7b	模型位於這個容器的 volume。
127.0.0.1:11434 的 /api/tags 是空的	Python 連到的服務不是持有模型的容器。
Docker ps 發現另一個 ollama_local	它占用了 localhost:11434，但沒有模型。
JSON 沒有 response 欄位	API 回傳的是 error JSON，而非成功的 generate JSON。

診斷指令

```
docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Ports}}'
curl -s http://127.0.0.1:11434/api/tags
docker compose logs --tail 80 ollama
```

除錯原則：先確認『哪個服務在監聽 endpoint』，再確認『那個服務有沒有目標模型』，最後才檢查 Python。不要因為模型曾成功下載，就假設 endpoint 一定指向它。

6. Python API 呼叫：成功與失敗要分開

目前 verify_local.py 是最小可運作版本。下一次調整時，應加入 HTTP 狀態與 API error 的檢查；這樣錯誤訊息才會指向真正原因，而不是變成 KeyError。

```
response = requests.post(url, json=payload, timeout=30)
response.raise_for_status()
result = response.json()
```

```
if "error" in result:
    raise RuntimeError(result["error"])
```

```
print("模型回應:", result["response"])
```

檢查	它攔住的問題
response.raise_for_status()	HTTP 4xx 或 5xx，例如 endpoint 不存在、服務錯誤。
if 'error' in result	HTTP 200 但 API 以 JSON 回報錯誤，例如 model not found。
timeout=30	服務無回覆時，避免程式無限等待。
不要直接 result['response']	成功欄位不存在時，避免用 KeyError 遮蔽真實 API 錯誤。

7. 為什麼要先寫 Markdown 規格？

企業實作不一定全用 Markdown，也可能使用 Confluence、Notion、SharePoint、OpenAPI 或資料治理平台；但共通目的都是讓規則可討論、可版本控管、可驗收。

文件	避免的問題
problem_statement.md	需求一直膨脹，或做出主管其實不需要的功能。
kpi_dictionary.md	不同人用不同公式算成本率、離職率或耗損率。
management_policy.md	門檻、資料例外與權限規則只存在某個人的腦中。
SKILL.md	AI 系統可做與不可做的邊界不清楚。
acceptance_cases.md	工程、測試與業務各自認為『完成』的定義不同。

注意：文件不是一次寫到永遠正確。實務是先建立 v0.1，實作時發現缺少單位換算、未關帳規則或權限細節，再以新版規格與測試更新。文件不是程式的替代品；程式、Schema 與測試才是實際執行規則的地方。

8. Day 1 複習清單

以下問題請先閉卷用自己的話回答，再回來核對。能講清楚比背指令更重要。

- 1. 為什麼下載模型不等於微調模型？
- 2. 為什麼 Docker container 有模型，Python 仍可能找不到模型？
- 3. Docker volume 解決的是什麼問題？
- 4. 127.0.0.1:11434 解決的是什麼風險？它不能證明什麼？
- 5. 為什麼 OLLAMA_NO_CLOUD=true 不等於整個系統完全離線？
- 6. 如何證明 Ollama 實際使用 GPU，而不是只看到 GPU 有記憶體使用？
- 7. KPI 的數字為什麼不能交給 LLM 自由計算？
- 8. 『事件與異常同期間發生』和『事件是根因』有何差別？
- 9. 預算為 0 時，預算差異率應如何處理？
- 10. 為什麼使用者自己填的 requester_role 不能直接當成授權依據？

Day 1 完成判定

驗收	目前狀態
本地模型推論	完成：Python 可呼叫 qwen2.5:7b。
GPU 驗證	完成：ollama ps 顯示 100% GPU。
模型版本追蹤	完成：已記錄完整 digest。
Docker volume	完成：已設定，另應保留重啟驗收證據。
Port 限制	完成：僅發布至 127.0.0.1。
完全斷網驗收	保留至 Day 7；目前不可宣稱已證明。
需求與規格文件	完成初稿；後續會隨實作修訂。

進入 Day 2 前的提醒：先建立虛構資料與資料品質檢查，再寫分析。不要從聊天介面開始，也不要讓模型替你算 KPI。資料口徑正確、可追溯，才是後續 RAG、Tool 與 Agent 的地基。